

CIA

VM2 Security Audit Report

Gitea Git Service

May 2026

Version 1.0

Team Members

Muhammad Noor Afaqi

Subhan Khaliq

Shanzay Asad

Table of Contents

Overview.....	3
1. Reconnaissance	3
1.1 Port Scan	3
1.2 Gitea Enumeration.....	3
1.3 Network Topology.....	4
2. Vulnerabilities.....	4
1. Gitea 1.4.0 Unauthenticated RCE (CVE-2018-1000003)	4
2. Root Credential Exposed in Leaked Credential File	5
3. All Application Secrets Exposed in Docker Container ENV	6
4. Dirty COW Local Privilege Escalation (CVE-2016-5195)	6
5. Open Registration on Gitea.....	7
6. Container Compliance Failure: Not Running as service-web.....	7
7. Publicly Accessible Repository (Misnamed as Private)	8
8. SSH Root Password Login Accepted	8
3. Tools Used.....	9
4. Additional Recommendations.....	9

Overview

Field	Detail
Local IP	192.168.64.6
OS	CentOS Linux 7 (Core), kernel 3.10.0-327.el7.x86_64
Services	SSH:22 (key-only), Gitea:80 (Docker, port 3000→80)
Gitea Version	1.4.0 (Go1.9.4)
Docker Containers	m2_server_1 (Gitea), m2_db_1 (MySQL 5.7)
Date	27/05/2026
Tester	Techies
Status	Root access achieved

1. Reconnaissance

1.1 Port Scan

Command: `nmap -sV -p 22,80,3000,8080,9000 192.168.64.6`

Port	Finding
22/tcp	OpenSSH 7.4, port open can log in
80/tcp	Gitea 1.4.0, served via Docker container m2_server_1 (internal port 3000)
3000/tcp	Filtered externally, mapped internally via Docker to port 80
8080/tcp	Filtered
9000/tcp	Filtered

These are all the open ports found on this VM.

1.2 Gitea Enumeration

The following was discovered via unauthenticated API, web access, and post-exploitation inspection:

Finding	Detail
Gitea version	1.4.0
Go version	Go1.9.4
Registration	Anyone can register without invite or approval
Admin user	n0tth3adm1n (ID:1) with email: not.me@here.there
Admin password	Superpassword which was exposed in Docker container ENV
Public repos	n0tth3adm1n/NotAPublicRepo, publicly cloneable, contents benign
LFS_JWT_SECRET	c9AcR4j_6Vaehm7IduHuLDvjpjk5uVoj8ov_l2YDVo, exposed in app.ini
SECRET_KEY	amsosupersecret - weak, exposed in app.ini and container ENV
DB credentials	gitea:gitea (MySQL user), MYSQL_ROOT_PASSWORD=gitea — exposed in container ENV

1.3 Network Topology

The host runs two Docker containers on an internal bridge network. A subnet ping sweep revealed one additional live host at 192.168.64.5:

Interface / Network	IP / Range	Notes
enp0s1	192.168.64.6/24	Primary host NIC, shared subnet with other VMs
docker0	172.17.0.1/16	Default Docker bridge, unused (no-carrier)
br-7dc875959842	172.20.0.1/16	m2_gitea Docker network
m2_server_1	172.20.0.2	Gitea container
m2_db_1	172.20.0.3	MySQL container

2. Vulnerabilities

1. Gitea 1.4.0 Unauthenticated RCE (CVE-2018-1000003)

Attribute	Detail
Severity	CRITICAL
CVE	CVE-2018-1000003 / EDB-44996
Location	http://192.168.64.6/n0tth3adm1n/NotAPublicRepo.git/info/lfs/objects

Attribute	Detail
Auth required	None, fully unauthenticated
Status	vulnerable

Gitea 1.4.0 contains an unauthenticated Remote Code Execution vulnerability via the Git LFS endpoint. The attack chain: (1) POST to /repo.git/info/lfs/objects with OID '....custom/conf/app.ini' creates a fake LFS object pointing to Gitea's config; (2) GET retrieves the real app.ini, leaking LFS_JWT_SECRET; (3) attacker forges a valid JWT as admin (ID:1); (4) race condition via oversized Content-Length PUT writes a fake session file; (5) forged cookie authenticates as admin; (6) admin creates repo with malicious git hook; (7) git push triggers hook, executing arbitrary OS commands.

Vulnerability confirmed via LFS probe returning an expires_at field in the 401 response body i.e. characteristic of the vulnerable code path. Root was obtained via credential leak before RCE was triggered, making execution unnecessary.

Patch:

- upgrade to Gitea >= 1.4.1; disable LFS if not required; restrict Gitea to internal network only.

2. Root Credential Exposed in Leaked Credential File

Attribute	Detail
Severity	CRITICAL
Location	Host OS /etc/shadow + leaked .creds.txt
Credential	root:admin
Status	EXPLOITED full root shell obtained via SSH password auth (contradicts key-only nmap finding, password auth was in fact accepted)
Impact	Complete host compromise, all container secrets readable, Docker socket accessible

The file .creds.txt, recovered during the VM1 phase, contained the credential pair root:doNotCommunicate. This was tested against SSH on 192.168.64.6 and granted an interactive root shell. This vector bypasses the RCE entirely and represents the highest-impact finding on VM2.

```
ssh root@192.168.64.6 => [root@localhost ~]#
```

Patch:

- immediately rotate all credentials listed in .creds.txt; enforce PermitRootLogin no and PasswordAuthentication no in sshd_config; distribute SSH keys to authorised operators only.

By then using hydra to use the credential list provided by the sysadmin we were able to go into VM2 at root with password pair easily.

Password used was admin.

3. All Application Secrets Exposed in Docker Container ENV

Attribute	Detail
Severity	CRITICAL
Location	docker inspect m2_server_1 / m2_db_1 — ENV block
Secrets exposed	GITEA_ADMIN_PASSWORD=superpassword, SECRET_KEY=amsosupersecret, LFS_JWT_SECRET=c9AcR4j_6Vaehm7IduHuLDvjpjk5uVoj8ov_l2YDVo DB_PASSWD=gitea, MYSQL_ROOT_PASSWORD=gitea
Status	all secrets read via docker inspect as root

Running docker inspect on both containers as root revealed all sensitive credentials in plaintext within the Env array. This included the Gitea admin password, JWT secrets, database credentials, and MySQL root password. Any user with access to the Docker socket or host root could extract these instantly.

```
GITEA_ADMIN_PASSWORD=superpassword
SECRET_KEY=amsosupersecret
LFS_JWT_SECRET=c9AcR4j_6Vaehm7IduHuLDvjpjk5uVoj8ov_l2YDVo
DB_PASSWD=gitea | MYSQL_ROOT_PASSWORD=gitea
```

Patch:

- use Docker secrets, environment-specific secret stores (HashiCorp Vault, AWS SSM Parameter Store), or encrypted .env files with restricted permissions. Never pass secrets as plain ENV variables in docker-compose.yml or Dockerfiles.

4. Dirty COW Local Privilege Escalation (CVE-2016-5195)

Attribute	Detail
Severity	CRITICAL (local privilege escalation)
CVE	CVE-2016-5195 (Dirty COW)
Kernel	3.10.0-327.el7.x86_64 — vulnerable (patched in 3.10.0-327.36.1)
Status	CONFIRMED VULNERABLE ; set up
Impact	Any local user could escalate to root by exploiting a race condition in the copy-on-write kernel mechanism

The host kernel 3.10.0-327.el7.x86_64 (CentOS 7, November 2015 build) is vulnerable to CVE-2016-5195 (Dirty COW). Just like VM1 which we found out previously.

Patch:

- yum update kernel — patched version is 3.10.0-327.36.1.el7 or later.

5. Open Registration on Gitea

Attribute	Detail
Severity	MEDIUM
Location	http://192.168.64.6/user/sign_up
Status	Exploited and account registered unauthenticated

Gitea allows account creation without any invite code, admin approval, or email verification. A test account was registered via a direct POST to /user/sign_up without authentication. This gives any external attacker a foothold on the platform, access to any repos visible to authenticated users, and a starting point for further authenticated attacks.

```
POST /user/sign_up => 302 redirect to /user/login (account created)
```

Patch:

- set `DISABLE_REGISTRATION = true` in `app.ini`, or if registration is required, enable `REQUIRE_EXTERNAL_REGISTRATION_TOKEN` and admin approval workflow.

6. Container Compliance Failure: Not Running as service-web

Attribute	Detail
Severity	MEDIUM, compliance failure
Requirement	Project spec mandates all containers run as user service-web
Actual	Containers run as root (<code>USER_UID=1000</code> set but no "User" field in Config — container process runs as git/root)
Status	Non-compliant

The project specification mandates all containers run under the user service-web.

The `docker-compose.yml` sets `USER_UID=1000` as an ENV variable, but does not configure the `user:` field in the service definition.

As a result, container processes run as the internal git user or root depending on the image entrypoint. This violates the deployment constraint and increases the vulnerability of the system in case of a container escape.

Patch:

- add `user: service-web` (or the corresponding UID) to each service block in `docker-compose.yml`; ensure the service-web user exists inside the container images with appropriate permissions on the data volumes.

7. Publicly Accessible Repository (Misnamed as Private)

Attribute	Detail
Severity	LOW
Location	http://192.168.64.6/n0tth3adm1n/NotAPublicRepo
Status	Identified — repo cloned unauthenticated (benign content)

The repository n0tth3adm1n/NotAPublicRepo is publicly cloneable without authentication despite its name. It is then clear someone accidentally named it private instead of setting it to be private

Current contents are limited to a LICENSE and README. The risk is low given benign content, but the misconfiguration pattern is notable and could recur with sensitive repos.

```
git clone http://192.168.64.6/n0tth3adm1n/NotAPublicRepo.git => success
```

Patch:

- set repository visibility to Private in Gitea settings; audit all repositories for unintended public access.

8. SSH Root Password Login Accepted

Attribute	Detail
Severity	MEDIUM
Location	SSH port 22
Note	SSH key-only auth appeared set — however root login via password succeeded, suggesting this was misconfigured or only applied at service level rather than via sshd_config

Overall a positive but misconfigured.

Initial nmap enumeration and a direct SSH attempt with no valid key returned 'Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password)', suggesting password auth was disabled. However, the root:admin credential from .creds.txt succeeded in opening a root shell.

This means either a misconfigured PAM stack, a per-user override, or that password auth is enabled but filtered differently at the GSSAPI negotiation level.

Patch:

- explicitly set PasswordAuthentication no and PermitRootLogin no in /etc/ssh/sshd_config and run systemctl reload sshd.
- Verify with: `sshd -T | grep -E 'passwordauth|permitroot'`.

3. Tools Used

Tool	Command	Purpose
nmap	nmap -sV -p 22,80,3000,8080,9000	Port scan and service detection
curl	curl -s GET/POST with custom headers	Gitea API enumeration, LFS vulnerability probe
git	git clone, git log, git branch -a	Repo cloning and history analysis
searchsploit	searchsploit gitea 1.4	CVE lookup — identified EDB-44996
ssh	ssh root@192.168.64.6	Root login with leaked credential
docker inspect	docker inspect m2_server_1 / m2_db_1	Extracted all secrets from container ENV
cat / find	cat app.ini, find / -name app.ini	Config file retrieval — LFS secret, DB creds
ping sweep	for i in seq 1 254; do ping -c1 ...	Subnet discovery — found 192.168.64.5
Python3 urllib	Cookie-aware POST loop	Credential brute force against n0tth3adm1n (unsuccessful)

4. Additional Recommendations

Beyond patching the identified vulnerabilities, the following general hardening measures are recommended for VM2:

Credential rework: all credentials in .creds.txt must be treated as fully compromised. Rotate every service credential, SSH key, and database password across all VMs. Implement a secret rotation policy.

Docker security posture: restrict Docker socket access to root only; consider enabling user namespace remapping (users-remap) to prevent container processes from running as host root; use read-only root filesystems where possible. Scan container images with Trivy or Gype as part of the CI pipeline.

Network segmentation: Gitea should not be directly accessible from the public network. Place it behind a reverse proxy with TLS and restrict access to internal subnets or VPN.

Kernel and OS patching: CentOS 7 with a 2015 kernel is severely out of date. Apply all available yum updates, with priority given to the kernel (Dirty COW) and any RHSA-rated critical advisories. Consider migrating to a supported OS (CentOS Stream, RHEL, Rocky Linux, AlmaLinux).

Monitoring and logging: no centralised log collection was observed. We can implement auditd rules to track privileged command execution, SSH logins, and Docker activity. Forward logs to the monitoring VM (Portainer/VM4)